

Java versus C e C++

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC

Slides inspirados nos slides da professora Manuela Sousa



Java vs C

Aspeto	Java	C
Tipo de linguagem	Linguagem orientada aos objetos	Linguagem procedimental (orientada às funções)
Unidade básica de programação	Classe (inclui estado e comportamento)	Função
Portabilidade	Sim, “ <i>write once, run anywhere</i> ”	Não. Recompilar para cada arquitetura/sistema
Compilador	<code>javac <filename>.java</code> Gera o ficheiro <code><filename>.class</code>	<code>gcc <filename>.c</code> (Linux), <code>cl <filename>.c</code> (Windows)
Execução	<code>java <filename></code>	<code>./<filename></code> (Linux), <code>./<filename>.exe</code> (Windows)
Pré-processador	Não	Sim
Convenção de <i>nomes</i> para os ficheiros com código fonte	Mesmo <i>nome</i> que o <i>nome</i> da classe; primeira letra de cada palavra do <i>nome</i> em maiúscula (e.g. <code>ArrayList.java</code>)	Todas as letras em minúsculas e palavras do <i>nome</i> separadas pelo carácter ‘ <code>_</code> ’ (e.g. <code>array_list.c</code>)
Inclusão de outros tipos	<code>import [<package>.]filename</code>	<code>#include “[<path>/]<filename>.h”</code>

Java vs C – memória

Memória	Java	C
Alocação de memória dinâmica	Operador new	Função malloc
Libertação de memória dinâmica	Não existe	Função free
Alocação de tipos compostos	Sempre no <i>Heap</i> com o operador new	<i>Heap, Stack e Data</i>
Endereços	Referência	Ponteiro
Operadores	Não tem	*, &, ->
Valor nulo	null	NULL
Tipo de dados genérico	Object	void *

Java vs C – tipos primitivos

Tipos primitivos	Java	C
Partilham praticamente os mesmos tipos: char, short, int, long, float, double Uma grande diferença é que o Java impõe dimensões pré-estabelecidas e independentes da arquitetura		
Booleano	. Tipo boolean . Não traduzido para inteiro: true e false	. Tipo bool em versões mais recentes . Traduzido para inteiro: true → 1 e false → 0 0 → false e diferente de 0 → true
Caractere	Codificado em Unicode com 16 bits	Codificado em ASCII com 8 bits
Tipos inteiros	byte (8 bits), short (16 bits), int (32 bits), long (64 bits)	A dimensão dos tipos inteiros depende da arquitetura e/ou da plataforma
Tipos com vírgula flutuante	float → 32 bits, norma IEEE 754 double → 64 bits, norma IEEE 754	float → 32 bits, norma IEEE 754 double → 64 bits, norma IEEE 754

Java vs C – *arrays* e *strings*

<i>Arrays e strings</i>	Java	C
<i>arrays</i>	. Tipo composto com sintaxe própria (indexação tal como no C) . Deteta indexação inválida (gera exceção) . Capacidade conhecida através do campo <code>length</code> . Objeto alocado sempre no <i>Heap</i> (com o operador <code>new</code>) ex: <pre>int[] a = new int[N]; int sizeA = a.length; // N</pre>	. Não deteta indexação inválida . Não conhece a capacidade . Alocado no <i>Heap</i>, <i>Stack</i> ou <i>Data</i> ex: <pre>int* a = (int*)malloc(N*sizeof(int));</pre>
<i>strings</i>	<code>String</code> -> tipo composto <i>built-in</i> da linguagem e imutável	Array de caracteres terminado por ‘ <code>\0</code> ’

Java vs C – funções/métodos (1 de 2)

Funções	Java	C
Funções/ Métodos	. Denominados por métodos . Enquadrados sempre dentro de uma classe (definem o comportamento do tipo composto)	. Denominado por funções (retornam valor) ou procedimentos (não retornam valor)
Assinatura	[<i>visibility</i>] [<i>static</i>] [<i>final</i>] <retorno> <nome> (<params>) . Pode ser marcado com o modificador de visibilidade <i>public</i>, <i>private</i> e <i>protected</i>; sem modificador explícito é <i>package</i> . Pode ser marcado com o modificador <i>static</i> tornando o método da <i>classe</i> em vez de <i>instância</i> . Pode ser marcado com o modificador <i>final</i> não permitindo a sua sobreposição em classes derivadas	[<i>static</i>] <retorno> <nome> (<params>) Pode ser marcado com o modificador de visibilidade <i>static</i> (torna-o privado ao módulo)
Convenção na definição do <i>nome</i>	Primeira letra do <i>nome</i> em minúscula e primeiras letras das restantes palavras que constituem o <i>nome</i> em maiúsculas ex: <code>numberOfItems()</code>	Palavras que constituem o <i>nome</i> em minúsculas separadas pelo caractere '_' ex: <code>number_of_items()</code>

Java vs C – funções/métodos (2 de 2)

Funções	Java	C
Passagem de parâmetros (argumentos)	Passados sempre por valor (cópia do valor): . Tipos primitivos por valor . Tipos compostos por cópia das referências (inclui <i>arrays</i>)	. Tipos primitivos passados por valor . Tipo <code>struct</code> passado por valor ou por ponteiro . <i>Arrays</i> passados por ponteiro
Número de parâmetros variável	<code><type> ... <paramName></code> ex: <pre>public PrintWriter printf(String format, Object... args)</pre>	<code>...</code> ex: <pre>int printf(const char * format, ...);</pre>
Variáveis locais	. Convenção na definição do <i>nome</i> igual à convenção usada na definição dos <i>nomes</i> dos métodos . Exige iniciação explícita (erro de compilação)	. Convenção na definição do <i>nome</i> igual à convenção usada na definição dos <i>nomes</i> das funções . Valores iniciais não determinísticos

Java vs C – estruturas de dados

Estruturas de dados	Java	C
Declaração	. Um ficheiro inclui a definição completa de uma classe . Os campos definem o estado do tipo ex: <pre>public class Point { public int x, y;</pre>	. Vários ficheiros podem incluir definições e declarações associadas a um tipo de dados (.c e .h) . Os campos definem o estado do tipo ex: <pre>struct point { int x, y; }</pre>
Acesso aos campos	<pre>Point pt = new Point(); pt.x = 0;</pre>	<pre>struct point pt; pt.x = 0; struct point* ppt = (struct point*) malloc(sizeof(struct point)); ppt->x = 0;</pre>
Iniciação automática dos campos	Campos e elementos dos <i>arrays</i> são iniciados: . 0 → tipos numéricos . false → booleanos . '\0' → caracteres . null → referências	Não é garantido
Métodos	Inclui tipicamente a definição de métodos que definem o comportamento do tipo	Não suporta a definição de funções

Java vs C – instruções de controlo de fluxo

Controlo de fluxo	Java	C
Partilham as mesmas instruções de controlo de fluxo: if-else; while; do-while; for; switch		
Instrução for-each	As instruções do corpo do ciclo são executadas para cada elemento dentro de uma coleção ex: <pre>int [] a = new int[N]; ... for (int e: a) { <executa N vezes e em cada iteração a variável e corresponde a um valor consecutivo do array a> }</pre>	Não existe
Instrução switch que retorna um valor	Introduzido a partir do JavaSE 12. ex: <pre><type> v = switch (<expressão>) { case Case_Label1, Case_Label11, ... -> <expressão>; case Case_Label2, Case_Label21, ... -> { <instrução>;* yield <expressão>; } default -> <expressão>; {<instrução>;* yield <expressão>; } }</pre>	Não existe

Java vs C e C++ – aspectos diversos (1 de 2)

	Java	C e C++
Comentários	Comentário de linha: // Comentário de bloco: /* ... */ Comentário JavaDoc: /** ... */ Os comentários JavaDocs são usados para documentar uma classe e os métodos da classe	Comentário de linha: // Comentário de bloco: /* ... */
Definição de um nome para um tipo que já existe	Não existe	typedef
Definição de tipos compostos	Apenas através de classes	<i>Classes, Structures e Unions</i>
Definição de variáveis globais	Não suporta	Suporta
Sobrecarga de operadores	Não suporta	Suporta (e.g. operador =, ==, >=, <, entre outros)
Operador '='	Copia referências	Copia o estado de um objeto alvo para um novo objeto

Java vs C e C++ – aspetos diversos (2 de 2)

	Java	C e C++
Herança múltipla	Não suporta a extensão de várias classes mas suporta a implementação de várias interfaces	Suporta
Tipo de dados genéricos em coleções	Através de genéricos	Através de <i>templates</i>
Construtor com um parâmetro do tipo da classe que está a ser criada	Equivalente a qualquer outro construtor	Denominado por <i>construtor por cópia</i> e usado para copiar o estado de um objeto alvo para um novo objeto
Destrutor de uma classe (libertação de memória)	Não suporta e a libertação é da responsabilidade do <i>Garbage Collector</i>	Suporta e a libertação é da responsabilidade do programador
Terminação de um programa	<code>System.exit(0);</code>	<code>exit(0);</code>

Java vs C – exemplo de um programa (hello world)

Java

```
import java.util.Scanner;

public class HelloWorld {
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);
        System.out.print(
            "Enter your name and press enter: ");
        String yourName = in.next();
        System.out.println(yourName +
            " says Hello World!");
    }
}
```

```
>javac HelloWorld.java
```

```
>java HelloWorld
```

```
Enter your name and press enter: Miguel
```

```
Miguel says Hello World!
```

C

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    char your_name[100];
    printf(
        "Enter your name and press enter: ");
    scanf("%s", your_name);
    printf("%s says Hello World!\n",
        your_name);
    return 0;
}
```

```
>gcc hello_world.c -o hello_world.exe
```

```
>hello_world.exe
```

```
Enter your name and press enter: Miguel
```

```
Miguel says Hello World!
```